- 新凯来笔试
  - 1. 坏了的计算器
  - 2.复原 IP 地址

新凯来笔试

1. 坏了的计算器

在显示着数字 startValue 的坏计算器上，我们可以执行以下两种操作：

双倍（Double）： 将显示屏上的数字乘 2；

递减（Decrement）： 将显示屏上的数字减 1 。

给定两个整数 startValue 和 target 。返回显示数字 target 所需的最小操作数。

```
// 逆向思维求解
class Solution {
    public int brokenCalc(int startValue, int target) {
        int res = 0;
        while(target > startValue){
                if(target % 2 == 0){
                        target /= 2;
                        res += 1;
                }
                else{
                        target += 1;
                        res += 1;
                }
        }
        if(target < startValue){res += startValue - target;}
        return res;
    }
}
```

2.复原 IP 地址

有效 IP 地址 正好由四个整数（每个整数位于 0 到 255 之间组成，且不能含有前导 0），整数之间用 '.' 分隔。 例如："0.1.2.201" 和 "192.168.1.1" 是 有效 IP 地址，但是 "0.011.255.245"、"192.168.1.312" 和 "192.168@1.1" 是 无效 IP 地址。 给定一个只包

含数字的字符串 s ，用以表示一个 IP 地址，返回所有可能的有效 IP 地址，这些地址可以通过在 s 中插入 '.' 来形成。你 不能 重新排序或删除。

示例 1：

```
输入：s = "25525511135"
输出：["255.255.11.135","255.255.111.35"]
```

示例 2：

```
输入：s = "0000"
输出：["0.0.0.0"]
```

示例 3：

```
输入：s = "101023"
输出：["1.0.10.23","1.0.102.3","10.1.0.23","10.10.2.3","101.0.2.3"]
```

```java
class Solution {
    List<String> ans = new ArrayList();
    List<String> t = new ArrayList();
    public List<String> restoreIpAddresses(String s) {
        dfs(s, 0, 4);   // 4表示需要有4段
        return ans;
    }

    public void dfs(String s, int i, int k) {
        if (k == 0) {
            if (i == s.length()) ans.add(String.join(".", t));
            return;
        }
        for (int j = i; j < s.length() && j < i + 3; ++j) {
            if (s.charAt(i) == '0' && j > i) return;    // 不能是前导0
            int v = Integer.parseInt(s.substring(i, j + 1));
            if (v >= 0 && v <= 255) {
                t.add(s.substring(i, j + 1));
                dfs(s, j + 1, k - 1);
                t.remove(t.size() - 1);
            }
        }
    }
}
```